

Projeto Blog Pessoal

O Projeto Blog Pessoal será nosso guia no aprendizado do framework ASP.NET Core e suas principais funcionalidades. Todo o código implementado no Projeto Blog Pessoal servirá de base para a construção do conhecimento adquirido nos cursos em andamento. O projeto receberá novas funcionalidades à medida que você adquirir os conhecimentos necessários através dos treinamentos.

O Projeto será composto por 3 Recursos (Conjunto de Classes e Interfaces responsáveis por mapear um tipo de Objeto e persistir no Banco de dados Relacional) e uma Classe auxiliar:

Classe	Descrição
Postagem	Recurso responsável por definir o Objeto Postagem (posts) do Blog Pessoal
Tema	Recurso responsável por classificar as postagens através do Objeto Tema
Usuario	Recurso responsável por definir o Objeto Usuário, que poderá acessar e criar postagens no Blog Pessoal
UsuarioLogin	Classe auxiliar, que será utilizada para efetuar login no Blog Pessoal

Cada Recurso irá gerar uma tabela no Banco de dados da aplicação. A Classe auxiliar não irá gerar uma tabela no Banco de dados da aplicação porque ela servirá de Classe auxiliar na implementação da Segurança da aplicação.

Os Recursos serão implementados na mesma sequência da tabela acima.

Antes de começar a criar as nossas Classes, vamos criar o nosso Projeto ASP.NET Core Web API no Visual Studio ou VS Code e configurar o Banco de dados da aplicação.

Este projeto permitirá consolidar conceitos de:

- Mapeamento de entidades;
 - Relacionamento entre tabelas;
 - Boas práticas de arquitetura em camadas;
 - Segurança de APIs RESTful;
 - Persistência de dados com Entity Framework Core;
 - Criação de APIs modernas utilizando ASP.NET Core.
-

Requisitos Funcionais

1. Gerenciamento de Usuários

- Cadastro de novos usuários.
- Atualização de informações de perfil.
- Exclusão de usuários.

2. Gerenciamento de Postagens

- Criação de novas postagens vinculadas a um usuário e a um tema.
- Atualização e exclusão de postagens existentes.
- Listagem de todas as postagens com suporte a filtros:
 - o Por tema.
 - o Por autor.

3. Gerenciamento de Temas

- Criação de novos temas.
- Atualização e exclusão de temas existentes.
- Listagem de todos os temas.

4. Autenticação de Usuários

- Login de usuários com validação de email e senha.
- Geração de tokens JWT para autenticação.
- Controle de permissões baseado no tipo de usuário.

Requisitos Adicionais com SonarQube

1. Análise de Qualidade de Código

- Configuração do SonarQube para análise de código no projeto.
- Integração com o build do projeto usando .NET CLI.
- Uso de métricas fornecidas pelo SonarQube, como cobertura de testes, complexidade e segurança.

2. Relatórios e Melhorias Contínuas

- Gerar relatórios de qualidade automaticamente após cada modificação no código.
- Resolver vulnerabilidades e aplicar melhorias com base nos insights fornecidos pelo SonarQube.

Requisitos Técnicos

1. Tecnologias Utilizadas

- Linguagem: C# 12+

- Framework: ASP.NET Core 8 Web API
- Banco de Dados: MySQL ou PostgreSQL
- ORM: Entity Framework Core
- Segurança: ASP.NET Core Identity + JWT
- Ferramentas: Visual Studio, VS Code ou Rider

2. Persistência de Dados

- Utilização do Entity Framework Core para gerenciamento das entidades.
- Relacionamentos entre entidades modelados no banco de dados:
 - Usuario 1:N Postagem.
 - Tema 1:N Postagem.

3. Arquitetura

- Arquitetura em camadas:
 - Controller: Gerencia as requisições HTTP e retorna as respostas.
 - Service: Implementa as regras de negócio.
 - Repository: Gerencia a interação com o banco de dados.
 - Model: Define as classes de entidade.
 - DTO (Data Transfer Object): Define os objetos de transferência de dados.

4. Segurança

- Senhas armazenadas com PasswordHasher.
- Controle de acesso aos endpoints com base no tipo de usuário.
- Uso de autenticação JWT.

Estrutura de Pastas do Projeto

BlogPessoal/

```

|
|— Controllers/
|— Services/
|— Repositories/
|— Models/
|— DTOs/
|— Data/
|— Config/
|— Middlewares/
|— Migrations/
|— appsettings.json
|— Program.cs
|— BlogPessoal.csproj
  
```

Endpoints da API

1. Usuário (/api/usuarios)

- POST /api/usuarios/cadastrar
 - Cadastrar um novo usuário.
- PUT /api/usuarios/{id}
 - Atualizar os dados do usuário.
- DELETE /api/usuarios/{id}
 - Excluir um usuário.
- POST /api/usuarios/login
 - Autenticar o usuário e gerar um token JWT.

2. Postagem (/api/postagens)

- POST /api/postagens
 - Criar uma nova postagem.
- PUT /api/postagens/{id}
 - Atualizar uma postagem existente.
- DELETE /api/postagens/{id}
 - Excluir uma postagem.
- GET /api/postagens
 - Listar todas as postagens.
- GET /api/postagens/filtro?autor={id}&tema={id}
 - Filtrar postagens por autor e/ou tema.

3. Tema (/api/temas)

- POST /api/temas
 - Criar um novo tema.
- PUT /api/temas/{id}
 - Atualizar um tema existente.
- DELETE /api/temas/{id}
 - Excluir um tema.
- GET /api/temas
 - Listar todos os temas.

Etapas de Desenvolvimento

Configuração Inicial

1. Dependências Necessárias

Instalar os pacotes NuGet:

```
dotnet add package Microsoft.EntityFrameworkCore
```

```
dotnet add package Microsoft.EntityFrameworkCore.Design
```

```
dotnet add package Pomelo.EntityFrameworkCore.MySql
```

```
dotnet add package Npgsql.EntityFrameworkCore.PostgreSQL
```

```
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
```

```
dotnet add package Swashbuckle.AspNetCore
```

2. Configuração do SonarQube

Criar um arquivo chamado sonar-project.properties na raiz do projeto:

```
sonar.projectKey=blog-pessoal-dotnet  
sonar.projectName=Blog Pessoal Backend ASP.NET Core  
sonar.host.url=http://localhost:9000  
sonar.login=<seu_token>  
sonar.sourceEncoding=UTF-8  
sonar.sources=.
```

Pipeline de Qualidade

- Integrar o SonarQube ao pipeline de CI/CD (caso aplicável).
- Configurar um estágio para análise de código e outro para geração de relatórios.

Configuração do Ambiente

Executando o SonarQube com Docker

```
docker run -d --name sonarqube -p 9000:9000 sonarqube
```

Executando a análise de código

```
dotnet sonarscanner begin /k:"blog-pessoal-dotnet" /d:sonar.login="SEU_TOKEN"
```

dotnet build

dotnet sonarscanner end /d:sonar.login="SEU_TOKEN"

Outras Configurações

1. Configuração Inicial

Criar o projeto ASP.NET Core Web API com suporte às seguintes dependências:

- ASP.NET Core Web API.
- Entity Framework Core.
- JWT Authentication.
- Swagger/OpenAPI.
- Banco de dados relacional (MySQL ou PostgreSQL).

2. Modelagem do Domínio

Criar as classes:

- Usuario
- Postagem
- Tema
- UsuarioLogin

Cada classe deve possuir:

- Propriedades;
- Relacionamentos;
- Validações;
- Configurações de persistência.

3. Implementação da Arquitetura

Criar as pastas:

- Models
- Repositories
- Services
- Controllers
- DTOs
- Config

- Data

Implementar:

- Repositórios;
- Serviços;
- Controllers;
- DTOs;
- Configuração do DbContext.

4. Segurança

Configurar:

- Autenticação JWT.
- Autorização baseada em Roles.
- Middleware de autenticação.
- Geração e validação de Tokens.

Criar um filtro para interceptar e validar o token JWT em todas as requisições protegidas.

5. Persistência e Validação

- Configurar o banco de dados.
- Criar as migrations.
- Gerar as tabelas automaticamente.
- Validar as entradas utilizando Data Annotations.

Exemplo:

[Required]

[StringLength(100)]

public string Titulo { **get**; **set**; }

6. Testes e Integração

- Criar testes unitários e de integração para validar as regras de negócio.
- Testar os endpoints com:
 - o Postman.
 - o Insomnia.
 - o Swagger.

7. Entrega Final

- Documentar os endpoints com Swagger/OpenAPI.
- Validar todas as funcionalidades.
- Revisar as boas práticas.
- Corrigir alertas identificados pelo SonarQube.

Critérios de Avaliação

1. Organização do Projeto

- Seguir os padrões de arquitetura propostos.
- Organização adequada das camadas.

2. Funcionalidade

- Todos os endpoints devem funcionar corretamente.
- As regras de negócio devem ser respeitadas.

3. Segurança

- Implementação segura de autenticação e autorização.
- Uso correto de JWT.

4. Documentação

- Endpoints documentados no Swagger.
- Código limpo e comentado quando necessário.

5. Boas Práticas

- Uso de DTOs.
- Validações.
- Controle de exceções.
- Padronização de respostas da API.
- Aplicação de Clean Code.

Objetivo do Projeto

O objetivo deste projeto é proporcionar aos estagiários uma experiência prática no desenvolvimento de APIs RESTful modernas utilizando ASP.NET Core, aplicando conceitos fundamentais de:

- Desenvolvimento Backend;
- Arquitetura em Camadas;
- Persistência de Dados;
- Segurança;
- Boas práticas de programação;
- Qualidade de Software.

O nível de dificuldade foi mantido equivalente ao projeto original em Spring Boot, permitindo que os alunos tenham uma experiência semelhante utilizando o ecossistema .NET.

DESAFIO:

Implementar IA usando os seguintes critérios:

Requisitos Funcionais

5. Recursos de Inteligência Artificial

Para introduzir conceitos iniciais de Inteligência Artificial de forma simples e didática, o projeto deverá implementar uma funcionalidade básica utilizando integração com API de IA.

Funcionalidades sugeridas

- Geração automática de resumo de postagens.
- Sugestão automática de títulos para postagens.
- Classificação automática de categoria da postagem.
- Moderação simples de conteúdo ofensivo.
- Geração automática de tags relacionadas ao texto da postagem.

Funcionalidade recomendada para os estagiários

Implementar uma funcionalidade chamada:

"Resumo Inteligente de Postagens"

Ao cadastrar uma postagem, a API deverá enviar o conteúdo da postagem para um serviço de IA, que retornará:

- Um resumo curto da postagem;
- Palavras-chave relacionadas;
- Uma sugestão de categoria.

Essas informações poderão ser armazenadas no banco de dados.

Objetivos pedagógicos da funcionalidade de IA

Com esta implementação, os estagiários irão praticar:

- Consumo de APIs externas;
- Programação assíncrona com `async/await`;
- Serialização e desserialização JSON;
- Criação de Services especializados;
- Boas práticas de integração;
- Tratamento de erros;
- Segurança no armazenamento de chaves de API.

Sugestões de APIs de IA

Os alunos poderão utilizar uma das seguintes APIs:

- OpenAI API;
- Gemini API;
- Azure AI Services;
- Hugging Face Inference API.

Exemplo de fluxo da funcionalidade

1. O usuário cria uma postagem.
2. A API recebe o conteúdo.
3. O Service de IA envia o texto para a API externa.
4. A IA retorna:
 - resumo;
 - palavras-chave;
 - categoria.
5. O sistema salva os dados no banco.
6. A postagem é retornada já enriquecida com informações geradas pela IA.

1. Gerenciamento de Usuários

- Cadastro de novos usuários.
- Atualização de informações de perfil.

- Exclusão de usuários.

2. Gerenciamento de Postagens

- Criação de novas postagens vinculadas a um usuário e a um tema.
- Atualização e exclusão de postagens existentes.
- Listagem de todas as postagens com suporte a filtros:
 - Por tema.
 - Por autor.

3. Gerenciamento de Temas

- Criação de novos temas.
- Atualização e exclusão de temas existentes.
- Listagem de todos os temas.

4. Autenticação de Usuários

- Login de usuários com validação de email e senha.
- Geração de tokens JWT para autenticação.
- Controle de permissões baseado no tipo de usuário.

Requisitos Adicionais com SonarQube

1. Análise de Qualidade de Código

- Configuração do SonarQube para análise de código no projeto.
- Integração com o build do projeto usando .NET CLI.
- Uso de métricas fornecidas pelo SonarQube, como cobertura de testes, complexidade e segurança.

2. Relatórios e Melhorias Contínuas

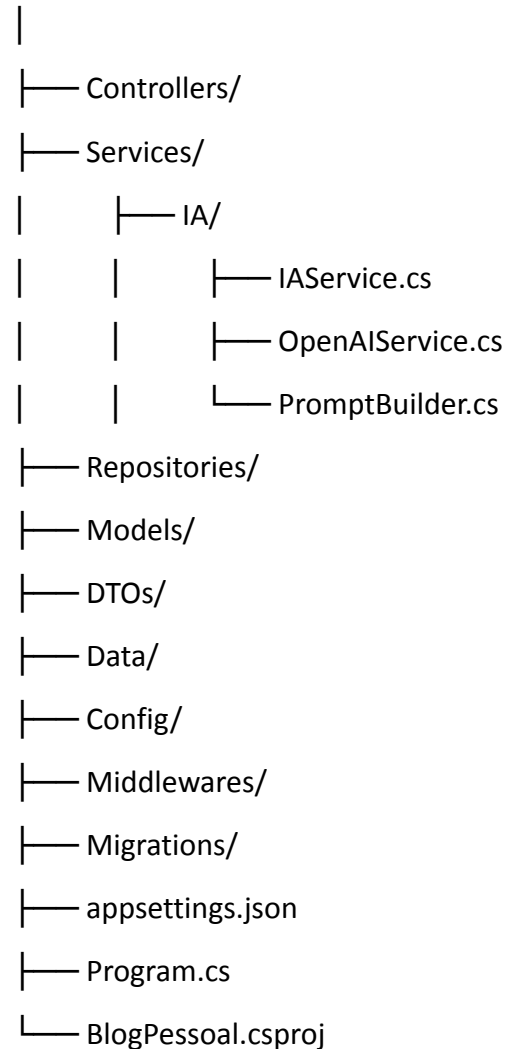
- Gerar relatórios de qualidade automaticamente após cada modificação no código.
 - Resolver vulnerabilidades e aplicar melhorias com base nos insights fornecidos pelo SonarQube.
-

Requisitos Técnicos

Integração IA: OpenAI API, Gemini API ou Azure AI Services

Estrutura de Pastas do Projeto

BlogPessoal/



Estrutura adicional da funcionalidade IA

A pasta IA dentro de Services será responsável por:

- Comunicação com APIs externas;
- Construção de prompts;
- Tratamento das respostas da IA;

- Regras relacionadas à inteligência artificial.

Novos Campos na Entidade Postagem

A entidade Postagem poderá receber os seguintes campos:

```
public string? ResumoIA { get; set; }
```

```
public string? TagsIA { get; set; }
```

```
public string? CategoriaIA { get; set; }
```

Esses campos serão preenchidos automaticamente após a análise do texto pela IA.

Novo Serviço de Inteligência Artificial

Criar uma interface:

```
public interface IIAService  
{  
    Task<ResultadoIA> GerarResumoAsync(string conteudo);  
}
```

Criar uma implementação:

```
public class OpenAIService : IIAService  
{  
}
```

Novo DTO

Criar um DTO chamado:

ResultadoIA

Com os seguintes atributos:

```
public class ResultadoIA
{
    public string Resumo { get; set; }

    public string Tags { get; set; }

    public string Categoria { get; set; }
}
```

Configuração da Chave da API

Adicionar no arquivo appsettings.json:

```
"OpenAI": {
  "ApiKey": "SUA_CHAVE_AQUI"
}
```

A chave nunca deve ser salva diretamente no código-fonte.

Novo Endpoint com IA

IA (/api/ia)

POST /api/ia/resumir

Responsável por:

- Receber o texto da postagem;
- Enviar para a IA;
- Retornar:
 - resumo;
 - categoria;
 - tags.

Exemplo de retorno:

```
{  
  "resumo": "Postagem sobre boas práticas em APIs REST.",  
  "categoria": "Tecnologia",  
  "tags": "API, REST, ASP.NET Core"  
}
```